IEEE Transactions on Quantum Engineering

Received 12 June 2023; revised 21 November 2023; accepted 22 November 2023; date of publication 28 November 2023; date of current version 18 January 2024.

Digital Object Identifier 10.1109/TQE.2023.3337328

Backtesting Quantum Computing Algorithms for Portfolio Optimization

GINÉS CARRASCAL[1,2], PAULA HERNAMPEREZ[3], GUILLERMO BOTELLA[4] (Senior Member, IEEE), AND ALBERTO DEL BARRIO[4] (Senior Member, IEEE)

[1]IBM Quantum, IBM Consulting España, 28830 Madrid, Spain
[2]Department of Informatic Systems and Computation, Faculty of Informatics, Complutense University of Madrid, 28040 Madrid, Spain
[3]CARTIF, 47151 Valladolid, Spain
[4]Department of Computer Architecture and Automation, Faculty of Informatics, Complutense University of Madrid, 28040 Madrid, Spain

Corresponding author: Ginés Carrascal (e-mail:gines_carrascal@es.ibm.com).

This work was supported in part by MCIN/AEI/10.13039/501100011033 under Grant PID2021-123041OB-I00, and in part by "ERDF A way of making Europe."
This work did not involve human subjects or animals in its research.

ABSTRACT In portfolio theory, the investment portfolio optimization problem is one of those problems whose complexity grows exponentially with the number of assets. By backtesting classical and quantum computing algorithms, we can get a sense of how these algorithms might perform in the real world. This work establishes a methodology for backtesting classical and quantum algorithms in equivalent conditions, and uses it to explore four quantum and three classical computing algorithms for portfolio optimization and compares the results. Running 10 000 experiments on equivalent conditions we find that quantum can match or slightly outperform classical results, showing a better escalability trend. To the best of our knowledge, this is the first work that performs a systematic backtesting comparison of classical and quantum portfolio optimization algorithms. In this work, we also analyze in more detail the variational quantum eigensolver algorithm, applied to solve the portfolio optimization problem, running on simulators and real quantum computers from IBM. The benefits and drawbacks of backtesting are discussed, as well as some of the challenges involved in using real quantum computers of more than 100 qubits. Results show quantum algorithms can be competitive with classical ones, with the advantage of being able to handle a large number of assets in a reasonable time on a future larger quantum computer.

INDEX TERMS Backtesting, conditional value at risk variational quantum eigensolver (CVaR VQE), portfolio optimization, quantum computing (QC), variational quantum eigensolver (VQE).

I. INTRODUCTION

Throughout history, humans have used tools to solve complex problems. As the complexity of problems has increased, so has the processing capacity of computers. However, there are still many problems that current computers cannot solve. Gordon Moore's prediction that the number of transistors in a microprocessor would double every two years has held true and has helped increase the processing capacity of computers. However, this trend is expected to end in the near future due to limits on the size of transistors [1].

The new generation of computers, quantum computers, still under development, promises a drastic advance in computing development as quantum computing (QC), for certain classes of problems, has the potential to drastically decrease computational cost and execution time [2], [3]. For this reason, the study of this new generation of computers is currently of great interest.

One of these complex problems is *portfolio optimization*. It is the process of selecting the best mix of investments to maximize return and minimize risk. This can be done by using a variety of techniques, including mathematical modeling, statistical analysis, and machine learning.

One research approach to solving complex financial services problems is to use QC algorithms [4]. These algorithms can help identify patterns and relationships that would be difficult to find with traditional methods [5]. By backtesting these algorithms, it is possible to see how they would have performed in different market conditions. This information can then be used to make better investment decisions in the future.

© 2023 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

Traditional portfolio optimization methods are based on classical algorithms that are not well suited to handle the large amounts of data involved in modern financial portfolios [6]. Quantum computers, with their ability to harness quantum superposition and entanglement to execute many computational paths simultaneously based on states correlation (quantum parallelism [7]), offer the promise of more efficient and effective portfolio optimization. However, QC also faces some challenges and limitations, such as noise, decoherence, and scalability.

QC is a new field that uses the principles of quantum mechanics to perform calculations that are beyond the reach of classical computers. It has the potential to solve complex problems, including some problems in finance and investment management. In the context of portfolio optimization, QC could be used to solve large or complex optimization problems, allowing for more accurate and sophisticated portfolio models and more informed asset allocation and risk management decisions. It could also be used to develop new optimization algorithms and techniques that take advantage of the unique properties of quantum systems, potentially providing more accurate and efficient solutions to portfolio optimization problems. While QC is still in the early stages of development, it has the potential to significantly improve portfolio optimization.

It is important to make it clear that today, quantum computers do not solve the portfolio optimization problem in a novel way and they do not reformulate the problem to make them easier to solve, instead, they solve the same optimization problem with different variable types, but in a different method.

The variational quantum eigensolver (VQE) is an algorithm for QC that is often used to solve optimization problems. It is based on the principle of quantum mechanics, which allows for the representation of complex mathematical functions as quantum states [8]. VQE is a hybrid quantum-classical algorithm that uses a quantum computer to find the lowest energy state of a system, which in this case would be the optimal portfolio [9].

To test whether VQE can outperform classical algorithms for optimization, researchers at IBM conducted a series of experiments on IBM quantum devices [10]. They found that VQE was able to find the global minimum in most cases and theoretically can outperform classical algorithms when the number of assets in the portfolio was large. While further research is needed to determine if VQE can be successfully applied to real-world financial portfolios, these results suggest that quantum computers may one day play an important role in helping us make better investment decisions.

Backtesting is crucial in evaluating the performance of a portfolio optimization algorithm. It involves using historical data or Monte Carlo simulations to determine the algorithm's ability to find the optimal portfolio and assess its robustness. When backtesting, one should consider using a large enough dataset, paying attention to results from different tests, and including a risk analysis to understand potential downsides. However, past performance does not guarantee future results.

The main contribution of this work is to establish a reusable methodology for backtesting of quantum and classical computing algorithms for portfolio optimization and using it to make 10 000 test runs, obtaining statistically significative results to compare three different classic algorithms and four quantum algorithms. We also address the challenges involved in using current real quantum computers of 27 and 127 qubits, especially the definition of an Ansatz with low length and the search for a suitable path in the real chip to map the circuit, in terms of quality and connectivity, to avoid a bigger circuit in the transpilation phase. Results show quantum algorithms can be competitive with classical ones, with the advantage of being able to handle a large number of assets in a reasonable time on a future bigger quantum computer.

The rest of this article is organized as follows. Section II introduces the idea of portfolio optimization. Section III explores the most important trends on current application of QC to portfolio optimization. Section IV explains the implications of quadratic programming. Section V describes the mathematical transformations needed to use QC for portfolio optimization. Section VI introduces backtesting as well as its benefits and drawbacks. Section VII describes our methodology based on backtesting comparison of different classical and quantum algorithms for portfolio optimization. Section VII details the implementation of the different algorithms to be comparable, and the considerations we take with real quantum computers. Section VIII presents the results. Finally, Section IX concludes this article.

Appendix A lists the various traditional methods that are used to solve this problem, and Appendix B describes the quantum algorithms that can be an alternative.

II. MARKOWITZ PORTFOLIO OPTIMIZATION

Modern portfolio optimization is the process of selecting the best mix of investments, for a given investor. To identify assets with low correlations and determine the optimal allocation of assets to maximize return on investment while minimizing risk. This selection must be made according to the investor's characteristics and risk aversion [11].

The goal of portfolio optimization is to find the optimal balance between risk and return. This can be a difficult task, as there are many factors that need to be considered, such as investment objectives, constraints, market conditions, and individual preferences. This typically involves analyzing the potential risks and returns of different assets, such as stocks, bonds, and real estate, and then creating a balanced portfolio that aligns with the investor's risk tolerance and financial goals.

The process of portfolio optimization involves several key steps. First, the investor must define their investment objectives, including their desired level of risk and return. This will help to determine the overall mix of assets that should

be included in the portfolio. Next, the investor must identify the various assets that are available for inclusion in the portfolio and analyze their potential risks and returns. Then, the investor must determine the optimal mix of assets that will form the portfolio. This typically involves using mathematical optimization techniques to find the combination of assets that are expected to provide the best return for a given level of risk. Finally, the investor must implement the chosen portfolio and monitor it regularly to ensure that it is still meeting the investment objectives. This may involve adjusting the portfolio, such as buying or selling assets, in order to maintain the desired level of risk and return.

While mathematical optimization, despite calculations being made using classical or quantum computers, can be a powerful tool for portfolio optimization, there are several limitations to be aware of. One major limitation is that optimization models are based on assumptions and historical data, and they may not accurately predict future market conditions [12]. In other words, the optimization process can only provide a best guess as to how a portfolio is likely to perform, and there is no guarantee that it will actually provide the expected return. Another limitation is that those optimization models are typically based on a static view of the market, which means that they do not consider any changes that may occur over time [13]. This can make it difficult for investors to adapt to changing market conditions and to respond to unexpected events.

Portfolio optimization is not an exact calculation valid for all investors, and has many external dependencies, and so it is difficult to determine which method is the best in a general sense.

III. STATE OF THE ART

As we delve deeper into the realm of portfolio optimization using QC, it is essential to understand the different approaches that researchers are exploring to tackle this challenge. These approaches can be broadly categorized into combinatorial formulations, convex formulations, and advanced formulations. By examining each of these categories, we can gain a comprehensive understanding of how QC techniques are being employed to enhance portfolio optimization and address various problem formulations.

Combinatorial formulations of portfolio optimization involve discrete variables, often representing fixed investment quantities or binary choices, such as whether to invest in a particular asset or not. These formulations typically present challenges in terms of scalability and complexity, making them suitable targets for QC techniques. Some of the notable QC methods being explored in this context include quantum annealing, quantum variational algorithms for NISQ computers, like VQE or quantum approximate optimization algorithm (QAOA), and Grover adaptive search (GAS) for fault tolerant quantum computers. Each of these techniques offers unique advantages, addressing different aspects of combinatorial portfolio optimization problems and different quantum computer capabilities.

As a brief overview of these pros and cons of the diferent algorithms, we can state that VQE has the advantage of using the shortest quantum circuits, enabling its use in current noisy real quantum computers. QAOA has potentially better convergence than VQE, but the circuits are longer, so it needs more error mitigation techniques that complicate the process. GAS has the advantage of a mathematically demonstrated quadratic speedup, but also has too long circuits to be reliable on computers without error correction, which complicate the process as well. In any case, these algorithms are the most used across the literature, and the more stable and tested, so we will center our backtesting experiment on them.

In contrast to combinatorial formulations, convex formulations of portfolio optimization involve continuous variables and linear constraints. These formulations are typically more amenable to traditional optimization techniques, but QC can still offer significant speedups and advantages in certain cases. Quantum interior-point methods (IPMs) for solving second-order cone programs and quantum linear systems (QLS) methods are two prominent QC techniques applied to convex portfolio optimization problems. These methods can potentially accelerate the solution process and provide more efficient results, but are far less developed and tested than the combinatorial optimization ones, making them a promising research target, but less suitable for fair backtesting comparison.

We have also advanced formulations since researchers are also exploring more complex portfolio optimization problems, such as mixed-integer programming and dynamic programming, which combine aspects of both combinatorial and convex formulations. These advanced formulations present unique challenges and opportunities for QC techniques. By adapting and extending the methods mentioned earlier, researchers are discovering innovative ways to apply QC to these advanced portfolio optimization problems. This ongoing work further expands the potential of QC in finance, revealing new possibilities for tackling a wide range of portfolio optimization challenges.

Two predominant computational models are employed by the research community: quantum annealing, which is realized by systems such as D-Wave, and Gate-based models, which are adopted by devices such as IBM.

The systematic review by Naik et al. [14] highlighted the diverse applications of QC in finance, ranging from portfolio optimization to blockchain technology, underscoring the broad potential and relevance of QC in various financial domains. This aligns with our focus on portfolio optimization, demonstrating the significance of QC in this area.

Egger et al. [15] provided a detailed exploration of quantum algorithms for finance-related applications, including simulation, optimization, and machine learning problems. This article not only complements our methodology in backtesting quantum algorithms, but also provides a broader context for the potential benefits and challenges of applying QC in financial services. Together, these papers form a solid foundation for understanding the current landscape and

future directions of QC in finance, directly supporting, and enriching the context of our research in portfolio optimization using QC.

A. COMBINATORIAL FORMULATIONS

Combinatorial formulations of portfolio optimization primarily focus on risk minimization, risk parity optimization, and mean-variance optimization.

Risk minimization [16] can be represented as a binary optimization problem. Tisi et al. [17] created a time dependant correlation graph to solve this problem by modeling price changes as daily data and thresholding the correlations to create sparsity in the graph, which is similar to (2) in this work. To solve the problem they used quantum annealing to find a maximum independent set in the constructed graph.

Rosenberg and Rounds [18] considered long-short minimum risk parity optimization and modeled whether to hold long or short positions using binary variables. They represented the problem as an Ising model, which is equivalent to a quadratic unconstrained binary optimization (QUBO) problem, and solve it with quantum annealing.

Mean-variance optimization can be formulated as a quadratic optimization problem with constraints, as seen in [19]. Hodson et al. [20] used the QAOA to solve a problem similar to (4) of this work, modeling three available strategies: stay long, stay short, or neither. They also model the constraints with mixing operators.

In a different approach, Gilliam et al. [21] employed a variation of GAS to solve (4) with a quadratic speedup over classical search in unordered data.

Rosenberg et al. [22] tackled time dependant versions of (4), where portfolio was optimized in different time points.

Mugel et al. [23] enhanced the model taking into account transaction costs and the disturbances in the market due to relatively big volume operations.

Grant et al. [24] provided benchmarking analysis for quantum annealers in portfolio optimization, while Venturelli and Kondratyev [25] proposed a reverse quantum annealing approach.

B. CONVEX FORMULATIONS

The original mean-variance portfolio optimization by Markowitz [19], [26], involve continuous variables and can be transformed into convex quadratic programs (QPs) with linear constraints without inequalities. The closed-form solution for this problem is given in [27]. However, more complex problems with additional constraints can benefit from advanced algorithms, such as interior-point methods, which can be solved with quantum IPMs) [28].

Rebentrost and Lloyd [29] presented a relaxed problem formulation that can be solved using QLS methods, potentially offering an exponential speedup over classical algorithms in some cases. They applied the Lagrange multipliers method and obtained a linear system, which can be expressed as a QLS. The solution of the QLSP can be obtained with time polylogarithmic complexity in terms of the system size.

C. OTHER APPROACHES

Heuristic-based quantum algorithms have also been applied to portfolio optimization. One example is the application of quantum walks to portfolio optimization by Slate et al. [30]. Quantum random walks can help explore the solution space more effectively compared to classical random walks. Quantum walks can be applied in both discrete and continuous time settings and have the potential to offer faster convergence to optimal solutions.

Some studies have focused on applying QC techniques for specific aspects of portfolio optimization, such as risk management. This includes the use of quantum algorithms for assessing portfolio risk, such as value at risk (VaR) and conditional value at risk (CVaR) [31], [32]. Quantum algorithms can potentially offer more efficient methods for estimating risk measures, which can be crucial for making informed portfolio decisions.

Meanwhile, the work in [33] offered practical insights into the application of quantum algorithms for stock price prediction, a closely related field to our work, showcasing the practicality and effectiveness of QC in real-world financial scenarios.

Alternative approaches for portfolio optimization include using quantum support vector machines (QSVMs) [34], which can provide a more efficient classification technique for portfolio selection. QSVMs can be used to identify assets that are likely to outperform or underperform the market, allowing for improved portfolio optimization.

IV. QUADRATIC PROGRAMMING

Quadratic programming is a mathematical optimization technique that is often used in portfolio optimization. It is a type of nonlinear optimization that involves minimizing or maximizing an objective function subject to constraints. It is an NP problem [35].

In the context of portfolio optimization, the objective function is typically a measure of portfolio performance, such as expected return or risk-adjusted return. The constraints typically include limits on the amount of each asset that can be included in the portfolio, as well as other factors such as the overall level of risk or the level of exposure to specific assets or sectors.

To solve a quadratic programming problem, an optimization algorithm is used to find the values of the variables (in this case, the amounts of each asset in the portfolio) that minimize or maximize the objective function, subject to the constraints. The resulting portfolio is then expected to provide the best possible performance, given the specified constraints. Quadratic programming has been widely used in portfolio optimization because it can handle a large number of variables and constraints, and it can provide solutions that are efficient and easy to interpret. It is also relatively fast and scalable, which makes it well-suited to the demands of modern investment management [36]. However, it has several limitations. It only works for differentiable objective

functions and constraints. Besides, it assumes that they are convex (i.e., have only one global minimum or maximum). This means that it may not be suitable for solving problems with discontinuous or nondifferentiable functions, or nonconvex problems. It is important for users to be aware of these limitations and use quadratic programming appropriately.

The number of assets that can be included in portfolio optimization using quadratic programming is limited by the computational power of the computer and the complexity of the problem. The exact number of assets that can be considered will depend on the specific implementation and available computing resources, but some quadratic programming solvers can handle problems with thousands of variables [37].

Quadratic programming algorithms may get stuck at a local minimum or maximum, rather than finding the global optimum. A local minimum (or maximum) is a point in the search space where the objective function has a lower (or higher) value than in the immediate surrounding area, but is not the global minimum (or maximum) of the function. This limitation can arise due to the specific optimization algorithm being used, as well as the nature of the objective function and the constraints. To try to avoid getting stuck at local optima, quadratic programming algorithms may use heuristic techniques such as random restarts or simulated annealing, but these techniques do not guarantee finding the global optimum. It is important for users to be aware of this potential limitation and use appropriate algorithms and heuristics to try to avoid it.

V. FROM QUADRATIC PROBLEM TO QUBO

Combinatorial optimization quantum algorithms work by representing the objective function and the constraints of the optimization problem as parameterized quantum states, and then using a classical optimizer to find the values of the variables (in this case, the amounts of each asset in the portfolio) that minimize or maximize the objective function, subject to the constraints.

A QP is a type of mathematical optimization problem that involves minimizing or maximizing a quadratic objective function subject to linear constraints. A QUBO problem is a similar type of optimization problem, but it involves binary variables (i.e., variables that can take on only two values, such as 0 or 1) and quadratic objectives without constraint functions.

Our problem is that combinatorial optimization quantum algorithms can solve a QUBO, not a QP.

In some cases, it is possible to transform a QP into a QUBO problem by replacing the continuous variables in the QP with binary variables. This can be done by defining a set of binary variables that represent the possible values of the continuous variables, and then using these binary variables in the objective and constraint functions of the QP.

For example, suppose we have a QP with a continuous variable x that can take on any real value. We can define binary variables, $x_0, \ldots, x_N$, that represent the possible values of x (i.e., $x_0 \ldots x_N$ can be the binary representation of x real value with certain precision). We can then rewrite the objective and constraint functions of the QP using the binary variables $x_0 \ldots x_N$, to create a QUBO problem.

As a general rule, the process of transforming a QP into a QUBO problem can be complex, but it can provide valuable insights and more efficient solutions to the optimization problem. By leveraging the unique properties of binary variables and quadratic functions, QUBO optimization can provide more accurate and efficient solutions to a wide range of optimization problems, including those in finance and investment management.

VI. BACKTESTING: AN ESSENTIAL TOOL FOR VALIDATION

Backtesting is a method of evaluating the performance of a trading strategy or investment portfolio by simulating its performance on historical data. It allows traders and investors to see how the strategy or portfolio would have performed under different market conditions and can be used to validate the effectiveness of the strategy or portfolio and to identify any potential weaknesses. To conduct a backtest, a trader or investor specifies the parameters of the strategy or portfolio and simulates its performance over a specific time period using historical data. Many authors use backtesting to show the benefits of their proposed algorithms or strategies [38], [39].

While backtesting can be a useful tool for evaluating and optimizing strategies or portfolios, it is important to note that past performance is not necessarily indicative of future results and that backtesting can be subject to biases that can affect the accuracy of the results.

Survivorship bias is a type of bias that occurs when a sample of data used in a backtest only includes assets that have survived until the present time and excludes assets that have been removed from the market or are no longer being traded. This can lead to an overestimation of the performance of the trading strategy or investment portfolio being tested, as it does not take into account the performance of assets that did not survive. To avoid survivorship bias, it is important to use a representative sample of data that includes both surviving and nonsurviving assets when conducting a backtest [40].

Look-ahead bias, also known as peeking or snooping bias, is a type of bias that occurs when a trading strategy or investment portfolio is evaluated using data that was not available at the time the decisions were made. In the context of backtesting, look-ahead bias can occur when the backtest includes data that were not available to the trader at the time the investments were made. This can lead again to an overestimation of the performance. Avoid look-ahead bias can help to provide a more accurate representation of the performance of the strategy or portfolio being tested [41].

As a result, backtesting can be a useful tool for evaluating and optimizing trading strategies and investment portfolios, but it is important to keep in mind its limitations and to use it

in conjunction with other tools and analyses. Although there is a work from Owhadi-Kareshk and Boulanger [42] describing a tool for backtesting classical and quantum annealing algorithms for portfolio optimization, they make an example of use for their software and not an actual analysis of the algorithms. To the best of our knowledge, this is the first work that performs a systematic backtesting comparison of classical and quantum portfolio optimization algorithms.

VII. PROPOSED METHODOLOGY

To compare and analyze different algorithms for portfolio optimization, we have defined several tests that enabled the addition of new algorithms in an streamlined way, making this methodology easily reusable.

The first test group analyzes the execution times of quantum and classical algorithms. The most important factor to consider is scalability.

We examine the possibilities of current large real quantum computers from IBM, identifying the strengths and the limiting factors.

In the second set of tests, we check which algorithm provides the best results, based on backtesting. To obtain statistically significant results, it is important to make hundreds of backtesting comparison experiments with different random subsets of stocks. In our case we have used ad base stocks from IBEX35 (benchmark stock market index of the Bolsa de Madrid, Spain's principal stock exchange) [43]. This methodology can be realized using any of the major stock indexes around the world, ensuring to take enough base stocks to ensure variability in the experiments.

Currently, quantum computers work with a queuing system in which users send their jobs to a quantum computer and wait for the computer to finish executing all the previous circuits, and then receive the results. Unfortunately, this way of working, when coupled with an iterative algorithm such as VQE, which sends a new request to a computer at each iteration, means that the execution of the program is prolonged.

The experiments in this work have been performed using Qiskit [44], a Python library to define and execute quantum circuits both on simulators and in real quantum devices. One interesting feature of Qiskit are Sessions. Sessions are a kind of minireservation of the system that avoids you waiting in the queue for each individual execution. However, this reservation is not exclusive. For big amount of circuit executions (like needed in backtesting experiments), with the sessions you get batches in the queue, but then other's jobs are interleaved to share the compute resources fairly.

There is also another factor impacting the total execution time: The communication times between the quantum computer and our own computer.

For these reasons, it has been decided to execute quantum algorithms, specifically VaR_VQE, on real quantum computers only for validation and illustrative purposes and make the quantum algorithms backtesting using simulators.

In any case, the methodology defined in this work is directly applicable to run on real quantum computers if dedicated resources and communications are available.

1) IMPLEMENTATION OF PORTFOLIO ALGORITHMS

The algorithms detailed in Appendices A (classical) and B (quantum) have been implemented in Python using the Qiskit library with different sets of historical adjusted closing price data from the IBEX35 stock market. These data have been taken from Yahoo! Finance [45] using the Python library *bt* [46].

Given a set of assets, we can build a portfolio by associating each asset with a (nonnegative) weight that will represent the proportion of total capital that has been invested in the aforementioned asset. Associated with a portfolio, there are the concepts of profitability and risk (sometimes we will refer to it as volatility) [19]. If we represent by μ the vector of expected returns of the considered assets and by Σ the matrix of variances and covariances between the assets, the return of the portfolio C with weights w will be given by the expression

$$\mu_C = \boldsymbol{\mu}^T \mathbf{w}$$

and its risk for

$$\sigma_C = \mathbf{w}^T \Sigma \mathbf{w}.$$

This study focuses only on portfolios that have equal weights for all their assets. We are interested in deciding which assets to invest in and which to avoid.

We will also impose the restriction that you can only invest in a maximum number of assets (budget). One way of expressing this in a matrix is to replace the vector of weights $\mathbf{w}$ by a binary vector $\mathbf{x}$ that verifies

$$\mathbf{1}^T \mathbf{x} = B \tag{1}$$

where B is the budget. Another approach is to use the vector of weights $\mathbf{w}$ and then pick a number B of assets randomly from among those with positive weight. We are indirectly assuming that all assets have the same cost and that with the value B together with (1), we have capital that we want to fully invest.

The algorithms, both classical and quantum, used in this work to solve this problem are based on heuristics, so the results depends on the data and some random components of the algorithms. This makes them difficult to compare. We cannot conclusively say that some are better than others. The traditional approach for comparing heuristic algorithms uses well known statistical tests for meaningfully relating the empirical performance of the algorithms and concludes that one outperforms the other. In contrast, authors like Perez et al. [47] build a predictive model of the algorithms behavior using functions that relate performance to problem size, in order to define dominance regions. Experimental results show that problem size affects differently diverse algorithms, in such a way that there could exist regions where one algorithm is more efficient than the other.

However, the classical algorithms considered in this study become inefficient since they require a large computational cost that grows exponentially with the number of assets considered for the construction of the portfolio.

A. CLASSICAL ALGORITHMS

1) MOVING AVERAGE STRATEGY (SMA)

Following this strategy, the moving average of the data is calculated. The assets whose closing price is higher than their moving average are considered. Finally, the desired amount of assets is chosen randomly among them to form the investment portfolio, weighted with equal weights.

The simple moving average strategy (SMAS) algorithm has been implemented with a time window of 50 days since most of the tests carried out have been done using similar timeframes.

2) RISK-RENTABILITY OPTIMIZATION (MVO)

To calculate the efficient frontier, we must solve for each value of R the problem (2).

$$\begin{aligned} &\min_{\mathbf{w}\in\mathbb{R}^n} \mathbf{w}^T \Sigma \mathbf{w} \\ &\text{s.t.} \\ &\mathbf{1}^T \mathbf{w} = 1, \\ &w_i \geq 0 \, \forall i, \\ &\mathbf{w}\boldsymbol{\mu} = R. \end{aligned} \tag{2}$$

The previous problem has a solution for each value of R, the expected return of the portfolio. For each valid R, we obtain a point of the efficient frontier. Any efficient portfolio is a good candidate solution. There are multiple criteria for choosing the most convenient efficient portfolio for the investor, and there are lines of study focused exclusively on this aspect.

Following the mean-variance optimization (MVO) algorithm, some efficient frontier portfolios are calculated and one of them is chosen using the cvxopt solver [48].

3) SHARPE RATIO OPTIMIZATION (SRO)

The problem of maximizing the Sharpe ratio is given by

$$\begin{aligned} &\max_{\mathbf{w}\in\mathbb{R}^n} \frac{\boldsymbol{\mu}^T \mathbf{w} - R}{\sqrt{\mathbf{w}^T \Sigma \mathbf{w}}} \\ &\text{s.a} \\ &\mathbf{1}^T \mathbf{w} = 1, \\ &w_i \geq 0 \, \forall i. \end{aligned} \tag{3}$$

To solve the Sharpe ratio maximization problem, the SRO has been implemented using function *calc_mean_var_weights* from package *ffn* (complement of the *bt* Python library).

B. QUANTUM ALGORITHMS

The formulation of the portfolio problem that we solve with the quantum algorithms is (4), where $\mathbf{x}$ will represent the binary vector of assets chosen to build our portfolio, $\boldsymbol{\mu}$ and Σ the return vector and the matrix of variances and covariances of the n assets considered, B the number of assets in which one wants to invest (budget), and q a positive term that measures the risk aversion of the investor.

$$\begin{aligned} &\min_{\mathbf{x}\in\{0,1\}^n} q\mathbf{x}^T \Sigma \mathbf{x} - \boldsymbol{\mu}^T \mathbf{x} \\ &\text{s.a} \\ &\mathbf{1}^T \mathbf{x} = B. \end{aligned} \tag{4}$$

The expression $\mathbf{1}^T\mathbf{x} = B$ is transformed into $(\mathbf{1}^T\mathbf{x} - B)^2$ and subtracted from the objective function of (4) multiplied by a penalty term. It must be taken into account that by performing this transformation we accept that the solution found may choose a number of assets different from the budget. The penalty term is the one that will measure how much it is acceptable to relax this restriction. For example, if we want to enforce the restriction, the penalty term will have to be greatly increased. By operating, we transform the expression in (4) into a quadratic form (5), for certain $\mathbf{A}$ and $\mathbf{b}$

$$\mathbf{x^T A x} + \mathbf{b^T x}, \;\; \mathbf{x} \in \{\mathbf{0}, \mathbf{1}\}^{\mathbf{n}}. \tag{5}$$

Performing the change of variable $x_i = (1 - z_i)/2$, for $z_i \in \{-1, 1\}$ in (5), we transform our problem into a new problem, which will consist of minimizing the expression (6), which coincides with the expression of the energy of the Hamiltonian of the Ising model. That way, our problem is equivalent to finding the lowest energy level of a Hamiltonian, which is possible using variational quantum algorithms, where $\mathbf{C}$ and $\mathbf{d}$ are the coefficients in the new domain.

$$\mathbf{z^T C z} + \mathbf{d^T z}, \;\; \mathbf{z} \in \{\mathbf{1}, -\mathbf{1}\}^{\mathbf{n}}. \tag{6}$$

The Hamiltonian of the Ising model is constructed by substituting in (6) the terms z_i by σ_z^i and the terms $z_i z_j$ by $\sigma_z^i \sigma_z^j$, obtaining a decomposition of the Hamiltonian as shown (10). Using Qiskit functions, we transform our problem into this new formulation in terms of Pauli Z-matrices. This formulation is common to all the quantum methods implemented.

The variational quantum algorithms have a quantum part and a classical part. The classical optimizer performs several iterations to calculate the parameters that optimize the problem and executes a circuit on a quantum computer in each iteration. Furthermore, for our type of problem, the use of the COBYLA optimizer is recommended [9], [49]. It is worth noting that COBYLA only performs one objective function evaluation per optimization iteration (and thus, the number of evaluations is independent of the parameter set's cardinality). Therefore, if the objective function is noise-free (simulators) and minimizing the number of performed evaluations is desirable, it is recommended to try COBYLA. We have also maintained the use of COBYLA on real noise quantum hardware for consistency and ease of results comparison. We

have seen similar results for COBYLA compared to other classical optimizers in nonfinancial use cases implemented on real hardware [50].

The execution time of a job in a quantum computer can be divided into communication time (between the classical computer and the quantum computer), validation time (verification of the sent job), queue time, and execution time. This last time comprises the time it takes to execute the circuit in the quantum chip and the measurements and postprocessing that are made to transform the values of the qubits into bits understandable by a classical computer. It must be taken into account that this execution time includes several executions of the quantum circuit and the corresponding measurements of the qubits after each execution of the circuit, in order to obtain the probabilities of the possible solutions.

In general, the validation time is negligible, being only a few milliseconds. In addition, the running time of a quantum circuit is also extremely short, so the running time is mainly the measurement time of the qubits. Finally, we comment that after carrying out various tests with real backends with different job numbers in the queue, it has been observed that communication times do not significantly affect the total execution time of the job. As such, queue time is by far the most time consuming. To avoid repeated queue waiting, Qiskit makes use of Sessions, avoiding the execution of a bunch of related circuits without waiting again in the queue. In the results of this study we have not considered queue times, we measure only execution time.

The stability of the current qubits does not reach the second [51], so the execution time of a quantum circuit (each shot) must not exceed that time in any case. With this in mind, we run the following tests to assess the possibilities of using this quantum portfolio optimization strategy on the largest available IBM real quantum computers.

During the preparation of this work (2022/2023), we utilized the most stable and proven 27-qubit machines from the Falcon family and the bigger experimental 127-qubit machines from the Eagle family.

1) VARIATIONAL QUANTUM EIGENSOLVER

To perform VQE, we run a parametrized Ansatz circuit in the quantum computer, and calculate the expectation value using the Hamiltonian: $\langle \boldsymbol{\phi}(\boldsymbol{\theta})|H_k|\boldsymbol{\phi}(\boldsymbol{\theta})\rangle$.

An advantage of this method to run on real hardware is that we can create low depth parametric circuits to be used as Ansatz suitable to run in the system without excessive noise. An example of the quantum circuit that VQE uses for five assets in the backtesting experiments can be seen in Fig. 2.

2) CONDITIONAL VALUE AT RISK VQE (VAR_VQE)

A variation of the VQE, changing the way of performing the calculations of the expectation value. It may slightly improve the classical postprocessing time, but has no implications in the depth of the quantum circuits.

3) QUANTUM APPROXIMATE OPTIMIZATION ALGORITHM

QAOA uses time evolution and layering to create a variational circuit and optimize its parameters. The algorithm concludes by sampling from the circuit to get an approximate solution to the optimization problem.

An example of the quantum circuit that QAOA uses for the five assets problem in the backtesting experiments can be seen in Fig. 1.

Comparing this circuit with the previously provided for VQE (Fig. 2), it is easy to observe that QAOA circuits have inherently more depth, making them more prone to noise disturbances on real computers. For this reason we have choosen VQE as the main algorithm for testing on real devices during this study.

4) GROVER ADAPTIVE SEARCH

This algorithm iteratively applies Grover search to find the optimum value of an objective function, by using the best known value from the previous run as a threshold.

This algorithm, based in Grover algorithm, inherits its quadratic speedup, but also inherits much longer circuits than the previously considered algorithms, making this algorithm more suitable for future fault tolerant quantum computers.

This algorithm is considered in this study at small scale with simulators, as we think it will be important in the future.

C. CONSIDERATIONS FOR 27-QUBIT MACHINES

Falcon quantum computers have a heavy hexagonal layout. This has to be taken into account to define a hardware viable Ansatz. For this experiment, we selected *ibm_cairo* as a 27-qubit machine (December 2022). The processor type is Falcon r5.11 (Multi-chip stack: The qubit chip is bump-bonded to an "interposer" chip that provides readout and signal delivery wiring). It has a Quantum Volume (QV) of 64 and can perform 2.4 K Circuit Layer Operations Per Second (CLOPS).

The system has the following native basis gates: CX, ID, IF_ELSE, RZ, SX, X.

Its median errors are as follows:

1) median CNOT Error: 1.049e−2;
2) median Readout Error: 1.180e−2;
3) median T1: 106.81 μs;
4) median T2: 102.88 μs.

The chip has the individual error distribution shown in Fig. 3. Observe that not all the qubits and connections are equally reliable, with bigger error rates on lighter-colored ones.

Before executing the quantum circuits we have to map the circuit qubits to the real hardware one, and also translate the original gates to the equivalent native ones.

To get the best results in the hardware we need to consider two important aspects: first, we need to use the qubits and links with better quality, and second, we need to select the mapping of the qubits to maximize the CNOT gates between

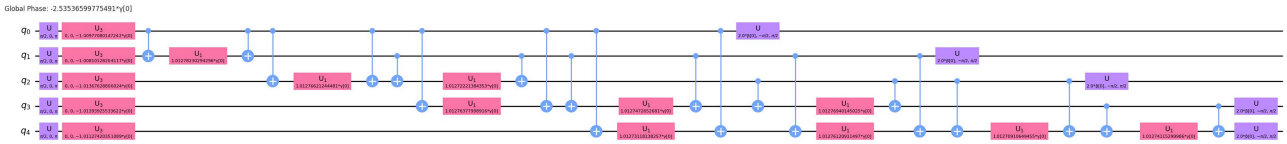

FIGURE 1. Example of Ansatz QAOA quantum circuit for five assets.

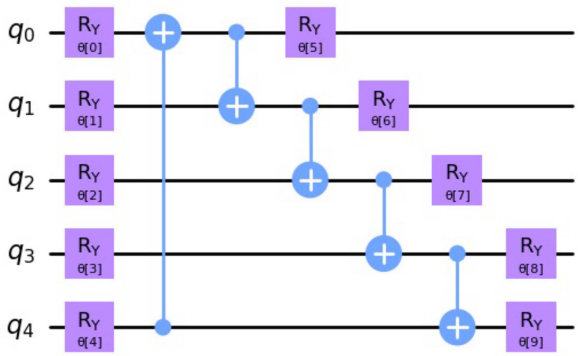

FIGURE 2. Example of Ansatz VQE quantum circuit for five assets.

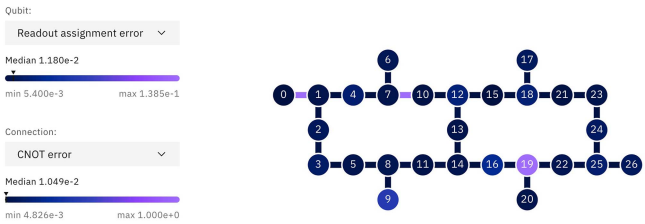

FIGURE 3. Layout and individual error distribution on *ibm_cairo"* real quantum computer.

real qubits with actual connection. If we need to perform CNOT gates between nonconnected qubits, we will need to add swap gates to move the needed qubits to a position in the chip with actual connection, generating a larger circuit.

We need to maintain the depth of the circuit to the minimum to avoid errors and to maintain our execution within the chip coherence time. If the errors grow up, the expectation values calculated became nearly random, and the classical optimizer would be unable to converge.

Avoiding qubits and CNOT links with bigger errors, we can find a linear path with a good performance of 19 qubits, as shown in Fig. 4.

We can create a VQE algorithm to select the best 5 out of 19 assets using this quantum computer. Randomly selecting 19 stocks from IBEX35 we get

ELE.MC, ITX.MC, FER.MC, CLNX.MC, ANA.MC, ENG.MC, IBE.MC, BBVA.MC, ACS.MC, VIS.MC, MTS.MC, GRF.MC, COL.MC, MEL.MC, ACX.MC, NTGY.MC, TEF.MC, SGRE.MC, SAN.MC.

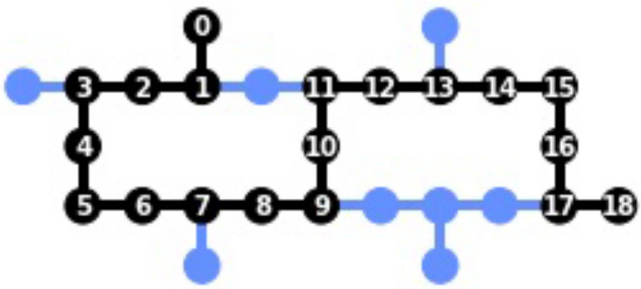

FIGURE 4. Optimum linear path with 19 qubits on *"ibm_cairo"* real quantum computer.

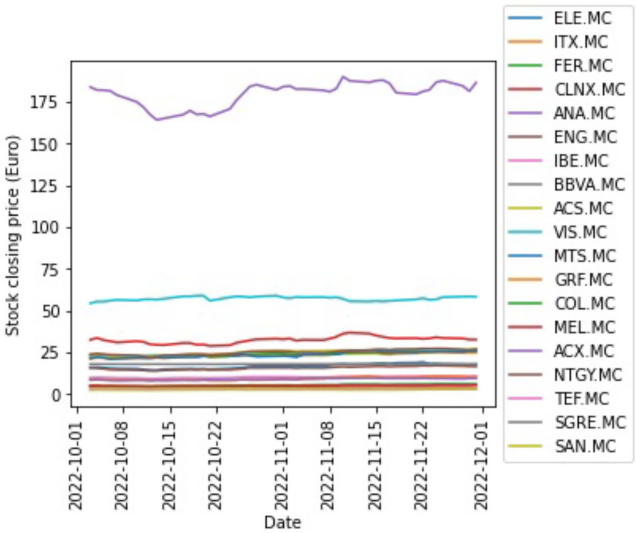

FIGURE 5. Evolution of closing price for the selected 19 stocks from IBEX35.

From Yahoo! Finance, we obtain the closing stock price for two months, from 2022-10-1 to 2022-12-1. The evolution of the stock prices can be seen in Fig. 5.

To make the optimization we create an Ansatz with linear entanglement that fits on the selected linear path and uses one qubit for each of the 19 stocks (Fig. 6).

D. CONSIDERATIONS FOR 127-QUBIT MACHINES

Eagle quantum computers have also a heavy hexagonal layout. This has to be taken into account to define a hardware viable ansatz. For this experiment, we selected

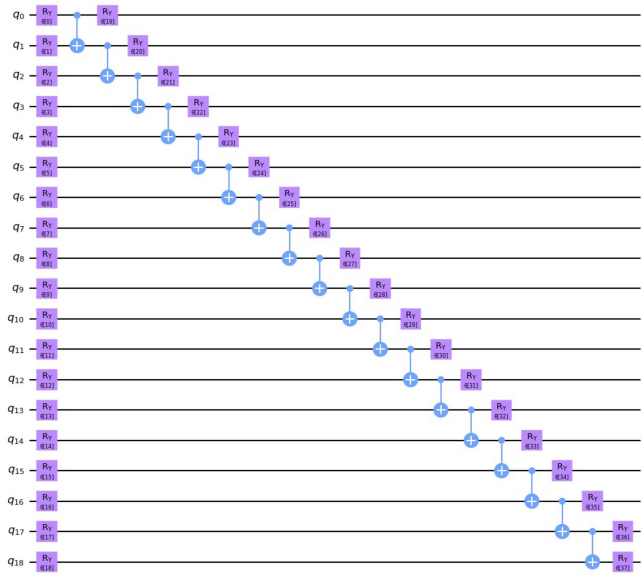

FIGURE 6. Linear entangled Ansatz VQE quantum circuit for 19 assets.

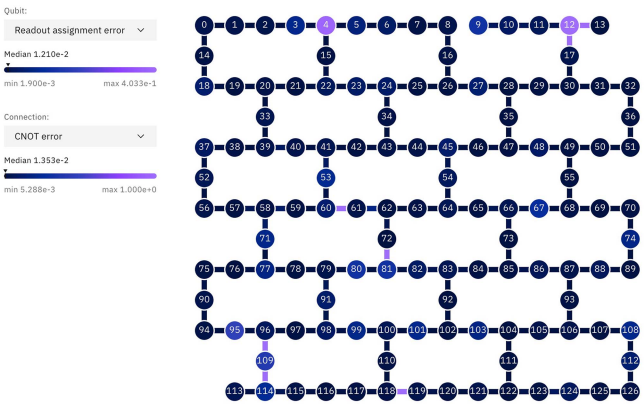

FIGURE 7. Layout and individual error distribution on *ibm_washington* real quantum computer.

ibm_washington" as a 127-qubit machine (December 2022). The processor type is Eagle r1 (increased scalability through novel packaging techniques). It has a QV of 64 and can perform 850 CLOPS.

The system has the following Basis gates: CX, ID, IF_ELSE, RZ, SX, X.

Its median errors are as follows:

1) median CNOT Error: 1.378e−2;
2) median Readout Error: 1.080e−2;
3) median T1: 99.55 μs;
4) median T2: 93.66 μs.

The chip has the individual error distribution shown in Fig. 7.

Avoiding qubits and CNOT links with bigger errors, we can find a circular path with an optimum performance of 28 qubits, as shown in Fig. 8.

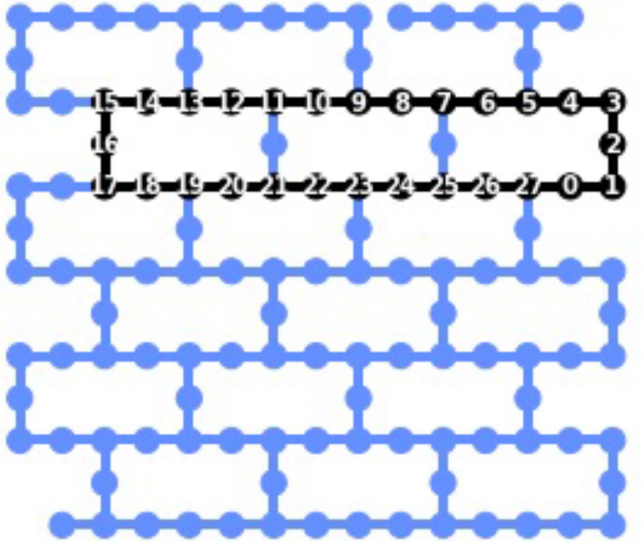

FIGURE 8. Optimum circular path with 28 qubits on *ibm_washington* real quantum computer.

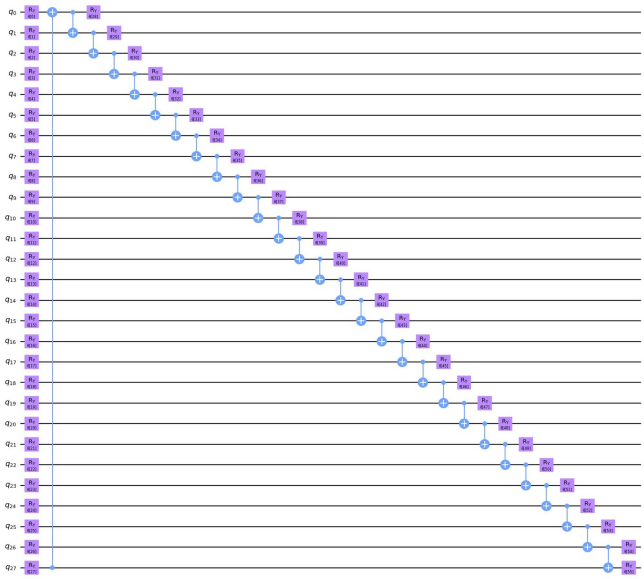

FIGURE 9. Circular entangled Ansatz VQE Quantum Circuit for 28 assets.

It is possible to find longer paths, but currently, this one will avoid an ansatz with depth near the coherence time of the chip.

We can create a VQE algorithm to select the best 5 out of 28 assets using this quantum computer. In order to do this, we create an ansatz with circular entanglement (Fig. 9).

VIII. RESULTS

A. BASIC EXECUTION 3 ASSETS ON REAL QUANTUM COMPUTER VERSUS CLASSICAL

Fig. 10 represents the obtained probabilities after executing a VQE with the data of three random assets from October

IEEE Transactions on Quantum Engineering

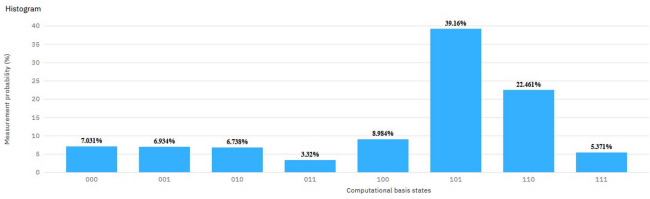

FIGURE 10. Probabilities obtained with VQE after 20 iterations on a real quantum computer (*ibmq_athens*).

TABLE 1. Optimal Results Obtained With the Three Asset Example Using Exact Calculations (NumPyMinimumEigensolver)

Choice	[111]	[011]	[101]	[001]
Values	4.0006	16.0000	16.0005	36.0000
Choice	[110]	[010]	[100]	[000]
Values	16.0001	35.9996	36.0004	64.0000

2020. For the execution of this instance, the five-qubit real backend *ibmq_athens* has been used, simply because it was the quantum computer with fewer tasks in the queue among those that meet the minimum requirements for the problem at that moment. In addition, the number of iterations of the VQE has been limited to 20.

Table 1 shows the exact values of the objective function for each possible choice of assets chosen. As can be seen, the result obtained with the algorithm is different from the optimal one, this is because 20 iterations are too few for the optimizer to find the desired minimum in this case and configuration.

B. EXECUTION TIME ON REAL QUANTUM COMPUTER VERSUS CLASSICAL

In this example, the approximate average time that each iteration has taken is two hours (including communication and queue times), however, these data is not really relevant, as it depends on many factors unrelated to the algorithm. As an observation and for context, it should be noted that there have been instances where circuits with double the qubits took approximately 44 per iteration. This significant time difference is attributed to the queue waiting time. In this particular case, the algorithm's iteration time was roughly nine seconds.

Although this time may seem quite long, the advantages of quantum computers over classical ones are that, presumably, this execution step time does not increase exponentially with the number of assets (qubits), which is the case with classical computers. To verify experimentally that this statement is correct, same steps of VQE has been executed with datasets with between 2 and 100 random assets and the average step execution time has been calculated in each case. These executions were carried out on three different 127-qubit backends: 1) *ibm_brisbane*; 2) *ibm_cusco*; and 3) *ibm_nazca* (real quantum computers). The results are shown in Fig. 11 . It is noticeable that the newer computers has not only better times, but also a smaller slope.

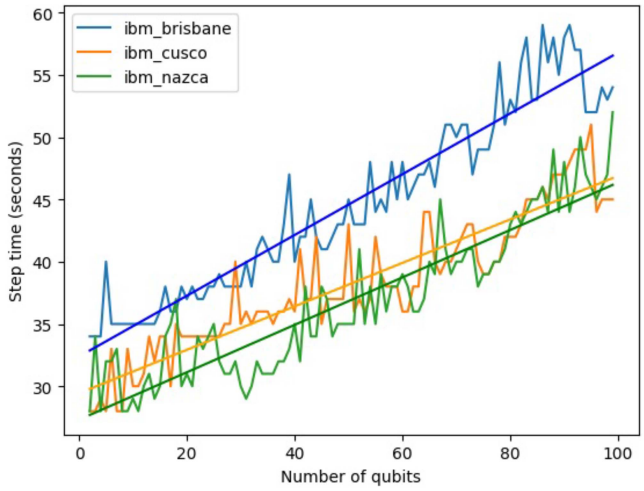

FIGURE 11. Execution time for one iteration step of quantum algorithms depending on the number of assets on different IBM real quantum computers (August 2023) (overimposed linear regression).

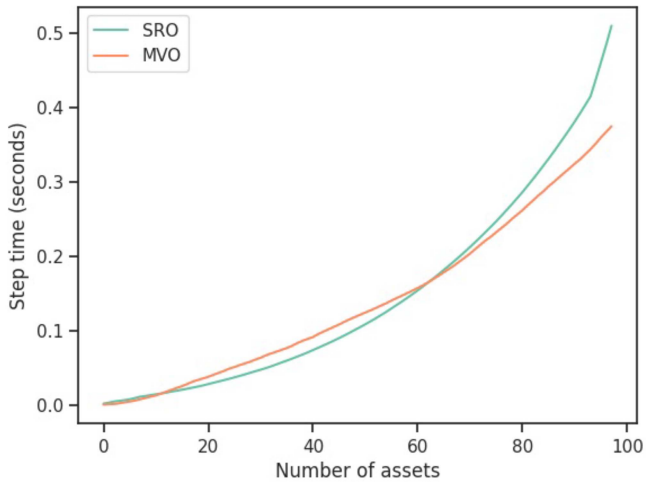

FIGURE 12. Mean execution time of one iteration step of classical algorithms depending on the number of assets (over 10 executions with random generated assets).

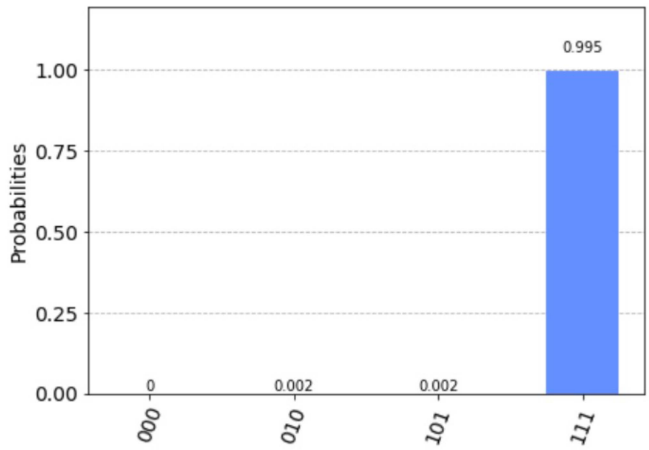

FIGURE 13. Probabilities obtained with VQE after 100 iterations on a simulator.

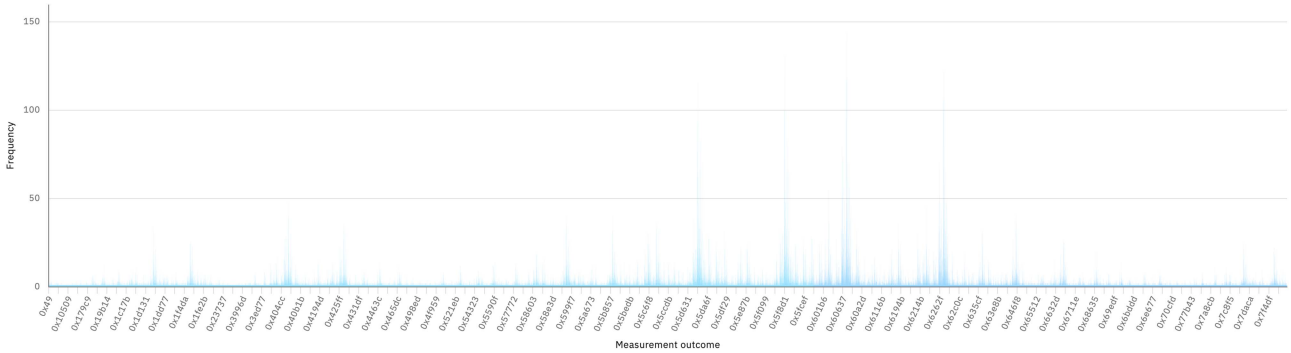

FIGURE 14. Example of probabilities measured on *ibm_cairo*, 27-qubit real quantum computer, to illustrate the exponential grouth in the number of states.

FIGURE 15. Linear entangled Ansatz VQE quantum circuit for 19 assets transpiled for *ibm_cairo* real quantum computer.

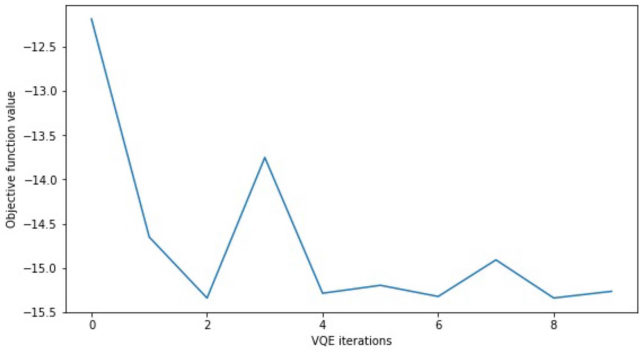

FIGURE 16. Convergence of CVaR VQE on *ibm_cairo*, 27-qubit real quantum computer.

To compare with the behavior of classical algorithms, the same experiment could be carried out as with the quantum. Fig. 12 represents the mean time taken for each step of the optimization. The exponential growth in the time is shown in this graph.

In the case of the SMAS, its computational cost is effectively linear, since the calculation of the MA is of np sums and p products, where p is the number of periods considered and n the number of assets with those who work. For this reason we consider that experimenting with this algorithm has no purpose for this exercise. As for SRO and MVO, both algorithms involve solving a quadratic programming problem, which is difficult to solve when real data is used together with extra constraints [52]. Both algorithms show an exponential trend. As the numbers grow larger, the exponential nature of the algorithms becomes more apparent. It is interesting to mention that SRO started having problems of feasability above 90 assets, so the times obtained are only relevant below this limit. Anyway, the SRO exponential behaviour is clearly shown.

In addition to the step time, the number of iterations needed for convergence with greater ammount of assets is unknown for both classical and quantum. More research should be necessary to identify possible differences of these algorithms at this respect. However, the exponential growth of the step time is enough to make the classical algorithms inefficient.

An alternative to using a real quantum computer is the use of a simulator. These simulators mimic the behavior of a quantum computer, making them very inefficient, but with the advantage of not having to wait in line or having the problem of communication times. In the previous example, we saw that the time for each iteration was huge, so the same problem has been solved with the simulator *ibmq_qasm_simulator* by allowing the optimizer to perform 100 iterations. The results are seen in Fig. 13. In this case, the desired optimum has been reached.

C. EXECUTING ON 27-QUBIT MACHINES

Fig. 15 shows the previously defined Ansatz circuit transpiled to the system's available gates and mapped to the selected 19 qubits, having a depth of 27 circuit layer operations

IEEE Transactions on Quantum Engineering

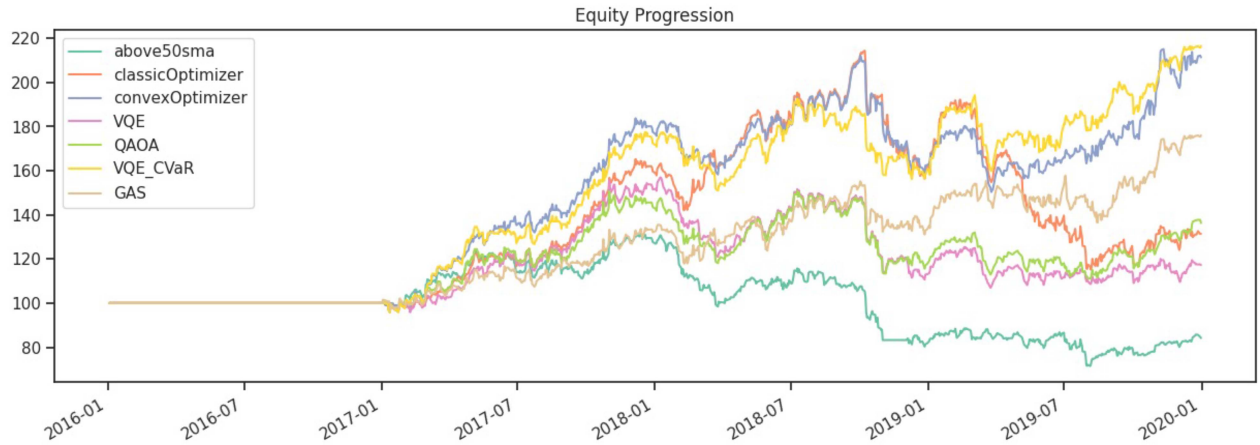

FIGURE 17. Example of one iteration of backtesting comparison of portfolio optimization with five random assets from IBEX35.

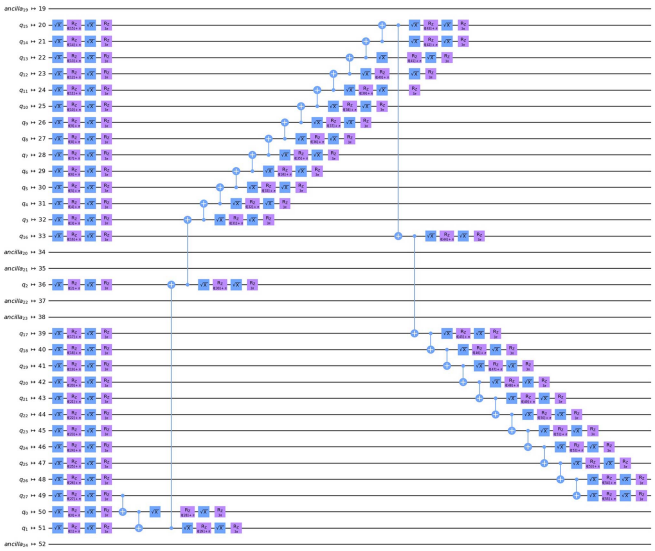

FIGURE 18. Circular entangled Ansatz VQE quantum circuit for 28 assets transpiled for *ibm_washington* real quantum computer.

TABLE 2. Gate Transpiling

Original gate	Transpiled gates
R_Y	$\sqrt{X} + R_Z + \sqrt{X} + R_Z$
$CNOT$	$CNOT$

on the real hardware. The Ansatz was selected this way so transpiling transforms the R_Y original gates only into $\sqrt{X}$ (blue) and R_Z (purple) gates, avoiding SWAP or any other more complex transformations. CNOT gates remain the same. Table 2 shows the transpiling strategy.

This takes a time of approximately 1125 μs on the quantum chip for each shot. The actual execution for 100 000 shots takes an average of 35.3 s in the quantum system (including quantum compute and near-time classical pre- and postprocessing. The queue time is not included.)

As an example, Fig. 14 shows the probabilities measured in the real quantum computer for one VQE iteration executed on *ibm_cairo* with 100 000 shots. The values in this graph are not relevant, they are used internally to calculate the expectation values of H. This figure is meant to show the growth of the number of different states in a quantum system when we increase the number of qubits.

The CVaR VQE algorithm converges very quickly and after ten iterations (Fig. 16) we obtain the following proposed optimal portfolio:

ITX.MC, BBVA.MC, ACS.MC, ACX.MC, SAN.MC.

We can now use the operator we built above regardless the specifics of how it was created. We set the algorithm for the NumPyMinimumEigensolver so we can have a classical reference. A backend is not required since this is computed classically not using quantum computation. It took 126.57 s to obtain the following results:

ITX.MC, BBVA.MC, ACS.MC, MTS.MC, SAN.MC.

As can be observed, it differs from the quantum result only in one of the selected stocks.

D. EXECUTING ON 127-QUBIT MACHINES

As shown in Fig. 18, this circuit transpiled to the system available gates and mapped to the selected 28 qubits has a depth of 37 Circuit Layer Operations on the real hardware. This will take an estimated time of 4352 μs on the quantum chip. The actual execution for 100 000 shots takes 48.6 s in the quantum system. The result is too big to be displayed as a histogram.

E. BACKTESTING

To analyze the performance of the algorithms, we prepare an experimental set with IBEX35 historical data from 2016 to 2020. As many algorithms rely on historical data, we left 2016 data for historical calculations, and started investing in 2017. Each strategy is permitted to calculate and rebalance the portfolio once a month. To be able to compare the results,

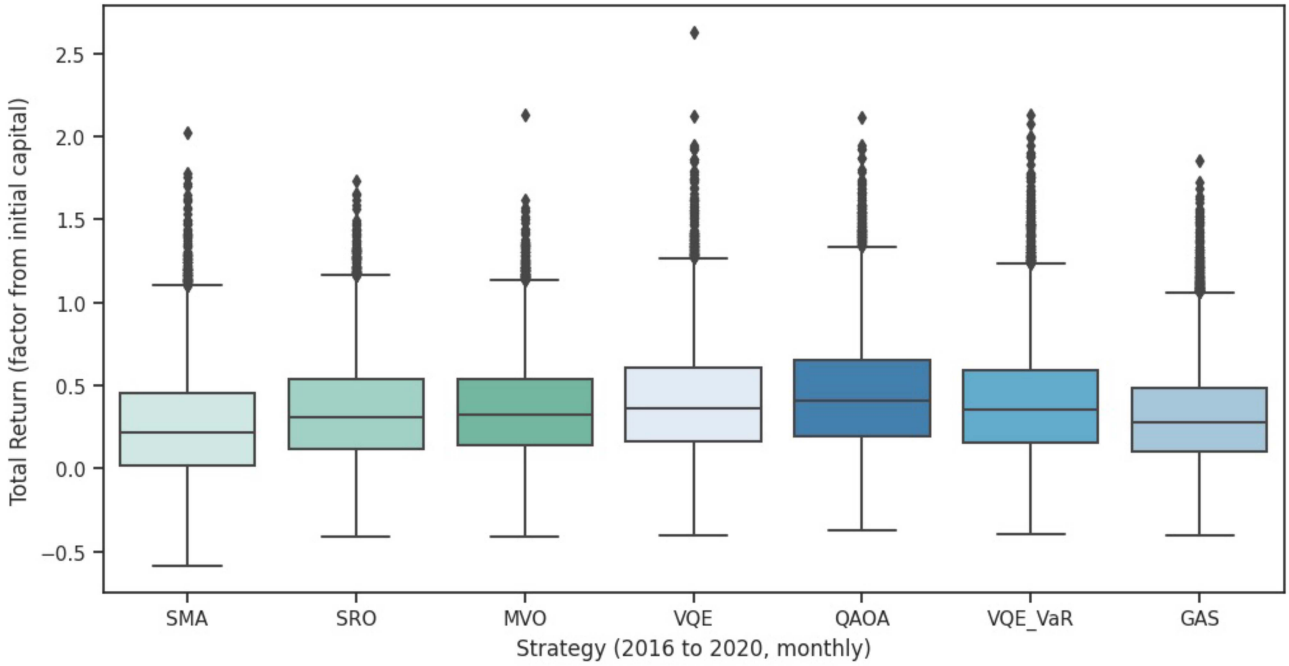

FIGURE 19. Boxplot of the main backtesting results.

TABLE 3. Main Statistics Obtained After the Execution of 10000 Simulations With Six Random Assets

Classical	SMA	SRO	MVO
mean	0.254676	0.345825	0.355986
std	0.328099	0.300733	0.299350

Quantum	VQE	QAOA	VQE_VaR	GAS
mean	0.402772	0.445264	0.395928	0.313526
std	0.331006	0.339129	0.333851	0.299147

TABLE 4. Number of Times That Each Algorithm Gets the Best Results (10000 Simulations With Six Random Assets)

Classical	SMA	SRO	MVO
Times Best	651	1753	1298

Quantum	VQE	QAOA	VQE_VaR	GAS
Times Best	1406	1783	1603	1506

the seven tested algorithms (SMA, MVO, SRO, VQE, VaR-VQE, QAOA, and GAS) were configured under the same conditions of risk aversion factor and avoided selecting two out of five available stocks. Then, the portfolio is rebalanced with the selected stocks equally weighted.

A suite of 10 000 backtesting experiments has been carried out with five different random assets from IBEX35. Each experiment provides a backtesting comparison of the performance of the different algorithms with the same data. An example of the results obtained on each experiment can be shown in Fig. 17.

The statistics of the results have been collected in Tables 3 and 4 where we can observe the mean and standard deviation of the results of each algorithm among the 10 000 simulations, as well as the number of times that each algorithm has given a better solution than the others. The high variance reflects the unpredictability of the stock market, where past events do not guarantee future outcomes. Fig. 19 shows the distribution of the total returns obtained with each strategy.

SMA performed poorly as expected, since it is a naive approach. We included SMA as a baseline to compare the other algorithms, both quantum and classic. Given that the rest of the algorithms have a very similar mean and a very large variance, it has been decided to filter the 10 000 executions based on the mean and variance of each execution.

We want to find out which algorithm performs better for a given set of assets. To do this, we need to compare the results of each algorithm for each record. A record is a set of stocks that we run the algorithms on. We use the intrarecord variance to quantify how much the results differ from each other for each record. A higher variance means that the results are more spread out and less similar. We are interested in the records that have a high variance, because they show us which algorithm stands out from the others. To filter out the low-variance records, we calculate the average of all the variances for all the records. Then, we only keep the records that have a variance higher than this average. These are the records that we use to compare the algorithms and find out which one is better. We check again how many times each algorithm has provided the best of the strategies. The results are collected in Table 5.

As can be seen, although initially, the quantum algorithms, except GAS, seemed to give slightly better results on more occasions than the classical ones; after filtering we verified that the results of the quantum algorithms are much better than the others on more occasions, highlighting especially

IEEE Transactions on Quantum Engineering

TABLE 5. Number of Times That the Algorithm Gets the Best Strategy (Filtered Simulations)

Classical	SMA	SRO	MVO
Times Best	193	528	380

Quantum	VQE	QAOA	VQE_VaR	GAS
Times Best	588	757	659	487

the QAOA and the VQE-CVaR. In any case, all the algorithms (both before and after filtering) always result in a better strategy than the others between 20% and 30% of the time, it is challenging to determine with complete certainty which algorithm outperforms the others.

The results shown in Tables 4 and 5, suggest that both the QAOA and the CVaRVQE are useful algorithms and to some extent competitive with the classical algorithms.

But the real advantage of using quantum computers for the portfolio optimization problem is not that the algorithm provides better or worse results, but that for a tremendously large number of assets, the classical algorithms become unfeasible. However, quantum algorithms would not have that problem. As long as there is a quantum computer with sufficient specifications, the VQE-CVaR algorithms will allow us to calculate a good solution to the portfolio optimization problem in a reasonable time.

IX. CONCLUSION

This work addresses a backtesting methodology of classical and QC algorithms for portfolio optimization. Results from backtesting the QAOA and CVaR-VQE algorithms suggest that they are useful and competitive with classical algorithms to some extent.

The scalability of current quantum computers from IBM is analyzed by testing their execution times. A second set of tests checks which algorithm provides the best results based on backtesting using historical data from the IBEX35 stock market in Spain. The algorithms were implemented using Python and the qiskit library.

For running on real quantum computers, we take into account the hardware design and qubit error distribution to find the best qubit mappings and gates to use. We also aim to keep the depth of the quantum circuits to a minimum to minimize errors and maintain execution within the coherence time of the chip. It has been used IBM backends of 27 and 127 qubits.

The backtesting results showed that, after filtering the experiments, both VQE and CVaR-VQE algorithms were competitive with the classical algorithms, with the advantage of being able to handle a large number of assets in a reasonable time on a future quantum computer.

Our models are still limited by the assumptions and simplifications that we have made to represent the stock market as a quantum system. Therefore, as future work, we plan to develop more realistic models that can capture the complexity and uncertainty of the real stock market conditions. The main problem to solve is how to select a portfolio of securities that maximizes the expected return and minimizes the risk, while taking into account the transaction costs and the integer constraints on the quantities of securities. Transaction costs are the fees that investors have to pay when buying or selling securities, such as commissions, taxes, or bid-ask spreads. Integer constraints are the restrictions that some securities can only be bought or sold in fixed units, such as shares or bonds. These factors make the portfolio selection problem more realistic, but also more challenging to solve. We also plan to implement error mitigation techniques to reduce the noise and errors that affect the quantum algorithms. Moreover, we plan to test our models on larger and more diverse datasets to validate their scalability and robustness.

APPENDIX A CLASSICAL ALGORITHMS FOR PORTFOLIO OPTIMIZATION

Next, we will explain three classical algorithms used to solve the portfolio problem in this work.

A. MOVING AVERAGE STRATEGY

A moving average [53] is a technical analysis tool that is used to smooth out price data by calculating the average price over a specified period of time. It is often used to help identify trends, spot trend reversals, and smooth out price fluctuations.

The concept of moving averages or MA refers to the calculation of the average in a time window of a time series. That is, for a time series, such as the closing prices of an asset, which we will denote x_t on each day t, the MA with a time window τ, is calculated as

$$\mathrm{MA}_t(X) = \frac{1}{\tau} \sum_{s=t-\tau+1}^{s=t} x_s.$$

For example, a 50-day SMA would be calculated by adding up the closing prices for the past 50 days, and then dividing the sum by 50.

For the implementation of the SMAS algorithm, the MA of each of the assets considered at the present moment is calculated. We will randomly choose the agreed number (budget) of assets from among those that are above their MA. These assets will be the ones that make up our investment portfolio, which we consider to have the same weight in the portfolio.

The SMAS algorithm is widely used in the financial world due to its easy understanding [54]. Moving averages can be used in various ways in technical analysis, such as identifying trends, spotting trend reversals, and identifying support and resistance levels. However, it is important to keep in mind that moving averages are a lagging indicator, meaning that they are based on past price data and may not always accurately predict future price movements.

B. RISK-RETURN OPTIMIZATION (MARKOWITZ MODEL)

The main theory on which portfolio optimization models are based is known as *modern portfolio theory*. This new approach was introduced by Markowitz who suggested analyzing both profitability and risk as a whole for the first time,

which caused a revolution in the financial world, giving rise to the new theory of portfolio optimization [19], [26].

This type of algorithm optimizes the risk-return ratio (MVO). Risk should be minimized and profitability should be maximized at the same time.

Profitability is defined as a financial ratio that measures the ability to generate profits, the measure of profitability will be the expected return of portfolio prices. The criterion that Markowitz's theory uses to choose the assets that make up the portfolio is their performance in relation to others. These must have low correlation between them and be diversified. This asset diversification process consists of dividing the investment among different stocks to reduce their overall exposure to risk without reducing the expected return.

Mathematically, a Markowitz model can be defined as follows:

$$\begin{aligned} &\max\ R_p - q\sigma_p^2 \\ &\quad \text{s.t.} \\ &x_i \geq 0, i = 1, \ldots, n \\ &\sum_{i=1}^{n} x_i = 1 \end{aligned} \tag{7}$$

where q is a risk aversion factor, R_p is the total return, and σ_p^2 is the variance of the portfolio. Besides, x_i are the amount invested in each asset, subject to budget and nonnegativity restrictions.

Prior to Markowitz's model, portfolio selection focused primarily on returns generated by investment opportunities. Markowitz maintains the emphasis on these, at the same time begins to consider risk as an important factor and the variance of a portfolio as a way of measuring risk.

Markowitz was the first to demonstrate how the variance of a portfolio can be reduced through the impact of diversification. It is proposed that investors focus on selecting portfolios based on their overall risk-reward rather than selecting portfolios that individually have attractive risk-return characteristics.

Markowitz's model accommodates the choice between risk and return of the investor. Regardless of the preference of the inverter, all these are under what is called the Markowitz efficient boundary. It is defined as the set of efficient portfolios, i.e., those that offer a higher expected return according to the different levels of risk that can be assumed.

This relates volatility and profitability of a portfolio, i.e., the benefits that the investor can obtain and the risks he must face to have it. This relationship must be positive, the higher the profitability, the larger the risk. At the border, risk-free assets are portfolios of minimum variance and are located to the left of the border (Fig. 20).

In the context of portfolio management, each investor must decide where to position. An investor with a conservative profile will opt for a maximum return and a minimum risk, however, someone riskier is positioned to the right of the border to obtain a higher return but with the possibility of suffering more losses.

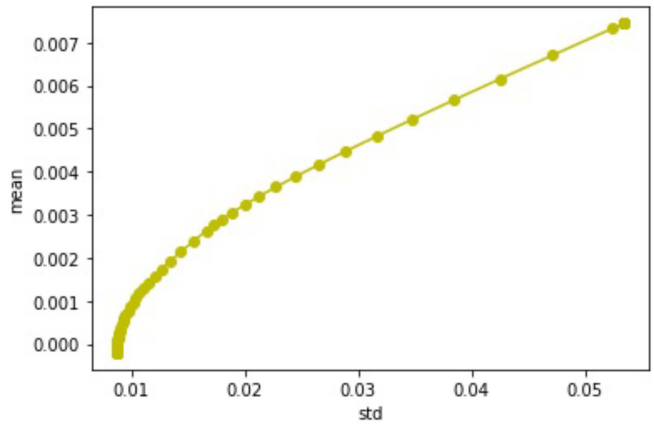

FIGURE 20. Graphical representation of the efficient frontier.

C. SHARPE RATIO OPTIMIZATION

The Sharpe ratio is a measure that allows the profitability of an investment portfolio to be related to its risk [55]. The Sharpe ratio is calculated by dividing the portfolio's excess return (the difference between its return and the risk-free rate of return) by its standard deviation, which is a measure of its volatility. It is calculated as

$$\text{SR} = \frac{\mu - R_f}{\sigma}$$

where μ represents the return of the portfolio made up of assets with strictly positive risk, R_f is the return of the asset without risk and σ the volatility of the portfolio.

A higher Sharpe ratio indicates a higher risk-adjusted return, meaning that the portfolio has generated a higher return per unit of risk.

The SRO algorithm is based on maximizing the Sharpe ratio to improve the return-risk ratio of our investment. The resulting portfolio is known as the market portfolio (optimal risky portfolio) [56].

This can be achieved through a variety of methods, such as diversifying the portfolio across a range of assets, using risk management techniques, or selecting assets with lower volatility.

It is important to note that while maximizing the Sharpe ratio may be a useful goal in some cases, it is not always the best approach to portfolio optimization. For example, an investor who is seeking to maximize the expected return of their portfolio may be willing to accept a higher level of risk in order to achieve higher returns. In these cases, maximizing the Sharpe ratio may not be the most appropriate goal [57].

APPENDIX B QUANTUM ALGORITHMS FOR PORTFOLIO OPTIMIZATION

The routines presented here are hybrid quantum-classical and compatible with each other.

A. VARIATIONAL QUANTUM EIGENSOLVER

The VQE algorithm is an algorithm used to calculate the smallest eigenvalue of a Hamiltonian, that is, its ground state.

Some of the properties of these matrices are the following.

1) The eigenvalues are always real.
2) The eigenvectors associated with distinct eigenvalues are orthogonal.

If H is a Hamiltonian (i.e., Hermitian matrix), and λ_i and $|\boldsymbol{\psi_i}\rangle$ with $i = 1, \ldots, n$ are their eigenvalues and corresponding unit eigenvectors, it is possible to represent the matrix as

$$H = \sum_{i=1}^{n} \lambda_i |\boldsymbol{\psi_i}\rangle\langle\boldsymbol{\psi_i}|. \tag{8}$$

The expected value of a matrix of H and a vector $|\boldsymbol{\phi}\rangle$ is defined as the value

$$\langle H\rangle_{\boldsymbol{\phi}} = \langle\boldsymbol{\phi}|H|\boldsymbol{\phi}\rangle = \sum_{i=1}^{n} \lambda_i |\langle\boldsymbol{\psi_i}|\boldsymbol{\phi}\rangle|^2. \tag{9}$$

It is evident, therefore, that for all $\boldsymbol{\phi}$, $\langle H\rangle_{\boldsymbol{\phi}} \geq \lambda_{\min}$ where $\lambda_{\min}$ is the smallest of the eigenvalues of H, since $|\langle\boldsymbol{\psi_i}|\boldsymbol{\phi}\rangle|^2 \geq 0$. Therefore, $\langle H\rangle_{\phi}$ are upper bounds of $\lambda_{\min}$ and also, if the eigenvalue $\lambda_{\min}$ is not double in the matrix, there will exist a unique eigenvector $\boldsymbol{\psi}_{\min}$ that verifies

$$\langle H\rangle_{\psi_{\min}} = \lambda_{\min}.$$

To find this eigenvector, the VQE algorithm uses what is known as a variational form or Ansatz. It is a circuit that is used to go iteratively from one estimate of $\psi_{\min}$ to another, seeking that in each assessment the expected value of the eigenvector is lower. There are different variational forms, but all depend on a series of parameters that grow with the number of qubits. Adjusting these parameters to the specific problem is done with a classical optimizer, hence the VQE is a hybrid algorithm.

Given a Hamiltonian H, of dimension $2^N \times 2^N$, which we can write in quantum form as a function of Pauli matrices $(\sigma_x, \sigma_y, \sigma_z)$ as in (10) where $i, j = 1, \ldots, N$; $\alpha, \beta \in \{x, y, z\}$; $h_i, h_{i,j} \in \mathbb{R}$. On one hand, σ_α^i refers to the tensor product of $N-1$ identity matrices of size 2×2 (σ_1) and the Pauli matrix σ_α at position i. On other hand, $\sigma_\alpha^i \alpha_\beta^j$ refers to the tensor product of $N-2$ matrices (σ_1) and σ_α at position i and σ_β at position j. For example, for $N = 3$, $\sigma_z^1 = \sigma_1\sigma_z\sigma_1$, and $\sigma_z^0\sigma_x^2 = \sigma_z\sigma_1\sigma_x$. We can rewrite H as a sum of K "elementary" Hamiltonians H_k, $k = 1, \ldots, K$ formed by a matrix of the type σ_α^i or $\sigma_\alpha^i\sigma_\beta^j$ multiplied by a real value

$$H = \sum_{i,\alpha} h_i \sigma_\alpha^i + \sum_{i,j,\alpha,\beta} h_{i,j} \sigma_\alpha^i \sigma_\beta^j. \tag{10}$$

The VQE for this Hamiltonian H works by minimizing (with a classical optimizer) the value of the function (11) where the calculation of $\langle\boldsymbol{\phi(\theta)}|H_k|\boldsymbol{\phi(\theta)}\rangle$ is performed by the quantum computer each time the function (11) is to be evaluated

$$f(\theta) = \frac{1}{K}\langle\boldsymbol{\phi(\theta)}|H|\boldsymbol{\phi(\theta)}\rangle = \frac{1}{K}\sum_{k=1}^{K}\langle\boldsymbol{\phi(\theta)}|H_k|\boldsymbol{\phi(\theta)}\rangle. \tag{11}$$

B. CONDITIONAL VALUE AT RISK–VARIATIONAL QUANTUM EIGENSOLVER (CVARVQE)

The VQE explained above seeks to reduce the mean (11). This would be the original approach of the algorithm; however, it is possible to consider other criteria when forming our asset portfolio.

Barkoutsos et al. [10] argued that an algorithm that gives a good approximation is better than an algorithm that tries to reach the exact solution and ends up giving a noisy result.

An optimization algorithm gives the best result found, not the exact one. Instead of choosing an objective function that tries to find the exact optimum, it is proposed to consider an objective function that allows us to find a better result avoiding the small, potentially noisy results.

With this idea, a better objective function for the optimization problem would be to find the minimum of a set of observations, $\min\{x_1(\theta), \ldots, x_K(\theta)\}$. However, this objective function is not adequate, since it is very poorly conditioned. Instead of this function, it is proposed to minimize CVaR.

Formally, the VaR of a variable X for a value $\alpha \in \{0, 1\}$ is defined as the smallest value x that verifies $F_X(x) \geq \alpha$ where F_X is the distribution function of the variable. And the CVaR is defined as the expected value, among the observations below the VaR, i.e.,

$$\mathrm{CVaR}_\alpha(X) = E[X|X \leq F_X^{-1}(\alpha)].$$

When we measure the qubits in a quantum circuit, the result is not a string of bits, but several strings accompanied by a probability. If we assume that $x_1(\theta), \ldots, x_r(\theta)$ are the observations obtained in a measurement with probabilities $p_0, \ldots, p_r$, ordered in such a way that $p_i \leq p_{i+1}$, our objective function for a fixed α will be

$$\mathrm{CVaR}_\alpha(\theta) = \frac{1}{\lfloor r\alpha\rfloor}\sum_{i=0}^{\lfloor r\alpha\rfloor} x_i(\theta)$$

where $\lfloor . \rfloor$ is the integer part function.

Using the CVaR as the objective function in our problem becomes a generalization between the objective function $f(\theta)$ of the previous VQE and $\min\{x_1(\theta), \ldots, x_K(\theta)\}$, since when $\alpha = 1$, $\mathrm{CVaR}_1(\theta) = E[f(\theta)]$ and $\lim_{\alpha\to 0^+} \mathrm{CVaR}_\alpha(\theta) = \min\{x_1(\theta), \ldots, x_K(\theta)\}$.

C. QUANTUM APPROXIMATE OPTIMIZATION ALGORITHM

Farhi et al. [58] introduced a quantum algorithm that produces approximate solutions for optimization problems, based on a variational approach that uses a sequence of unitary operators applied to an initial state. The algorithm depends on a positive integer p and the quality of the approximation improves as p is increased. The depth of the circuit

that implements the algorithm grows linearly with p times (at worst) the number of constraints.

The main idea behind the QAOA is to construct a quantum state that has a high overlap with the optimal solution of the problem. The circuit uses two types of unitary operators: 1) $U(C, \gamma)$, which depends on the objective function C and an angle γ, and 2) $U(B, \beta)$, which depends on a sum of single bit σ_x operators and an angle β. The objective function C is defined as the number of satisfied clauses in a combinatorial optimization problem, where each clause is a constraint on a subset of n bits. The operator $U(C, \gamma)$ is diagonal in the computational basis and its action is to increase or decrease the phase of each basis state depending on its value of C. The operator $U(B, \beta)$ is nondiagonal and its action is to create superpositions of basis states by applying σ_x to each bit with a probability depending on β. The initial state is chosen to be the uniform superposition over all basis states, denoted by $|s>$. The QAOA state for any integer $p \geq 1$ and $2p$ angles $\gamma_1, \ldots, \gamma_p = \gamma$ and $\beta_1, \ldots, \beta_p = \beta$ is defined as

$$|\beta, \gamma >= U(B, \beta_p)U(C, \gamma_p), \ldots, U(B, \beta_1)U(C, \gamma_1)|s > .$$

The goal is to find the optimal values of β and γ that maximize the expected value of C on this state, i.e.,

$$\max_{\beta,\gamma} < \beta, \gamma|C|\beta, \gamma > .$$

There are two methods to find these optimal values: One based on efficient classical preprocessing for fixed p, and another based on quantum adiabatic evolution for growing p. Acording to Farhi, QAOA can achieve better approximation ratios than classical algorithms.

D. GROVER ADAPTIVE SEARCH

Constraint satisfaction problems (CSPs) are a class of computational problems that involve finding assignments of values to variables that satisfy a set of constraints. CSPs are ubiquitous in many domains, such as artificial intelligence, cryptography, scheduling, and optimization. However, solving CSPs can be very challenging, as they are often NP-hard or NP-complete, meaning that there is no known efficient classical algorithm that can solve them in polynomial time.

Guilliam et al. [21] proposed a novel quantum algorithm for solving CSPs, called GAS. GAS is based on Grover's algorithm, which is a quantum search algorithm that can find a marked element in an unsorted database with quadratic speedup over classical search. GAS adapts Grover's algorithm to the CSP setting by using a quantum oracle that evaluates the constraints and marks the satisfying assignments. GAS also uses a feedback mechanism that dynamically adjusts the number of iterations of Grover's algorithm based on the estimated probability of finding a solution. GAS can solve any CSP with n variables and m constraints using $O(\sqrt{2^n/m})$ oracle calls and $O(n)$ qubits.

GAS is a promising quantum algorithm for solving CSPs and has potential applications in various other fields.

ACKNOWLEDGMENT

The authors would like to acknowledge the use of IBM Quantum services for this work. The views expressed are those of the authors and do not reflect the official policy or position of IBM or the IBM Quantum team.

REFERENCES

[1] M. M. Waldrop, "The chips are down for Moore's law," *Nature News*, vol. 530, no. 7589, pp. 144–147, Feb. 2016, doi: 10.1038/530144a.

[2] R. P. Feynman, "Simulating physics with computers," *Int. J. Theor. Phys.*, vol. 21, no. 6, pp. 467–488, 1982, doi: 10.1007/BF02650179.

[3] G. Carrascal, G. Botella, A. del Barrio, and D. Kremer, "A Bayesian-network-based quantum procedure for failure risk analysis," *EPJ Quantum Technol.*, vol. 10, pp. 1–19, Dec. 2023, doi: 10.1140/epjqt/s40507-023-00171-4.

[4] E. Yndurain, S. Woerner, and D. J. Egger, "Exploring quantum computing use cases for financial services, IBM Institute for Business Value, 2019. [Online]. Available: https://www.ibm.com/downloads/cas/2YPRZPB3.[dl:29.07.2020]

[5] J. Gacon, C. Zoufal, and S. Woerner, "Quantum-enhanced simulation-based optimization," in *Proc. IEEE Int. Conf. Quantum Comput. Eng.*, 2020, pp. 47–55, doi: 10.1109/QCE49297.2020.00017.

[6] R. O. Michaud and R. O. Michaud, *Efficient Asset Management: A Practical Guide to Stock Portfolio Optimization and Asset Allocation*, Oxford, U.K.: Oxford Univ. Press, 2008. [Online]. Available: https://global.oup.com/academic/product/efficient-asset-management-9780195331912

[7] A. Steane and E. Rieffel, "Beyond bits: The future of quantum information processing," *Computer*, vol. 33, pp. 38–45, 2000, doi: 10.1109/2.816267.

[8] D. A. Fedorov, B. Peng, N. Govind, and Y. Alexeev, "VQE method: A short survey and recent developments," *Mater. Theory*, vol. 6, no. 1, pp. 1–21, 2022, doi: 10.1186/s41313-021-00032-6.

[9] G. Buonaiuto, F. Gargiulo, G. De Pietro, M. Esposito, and M. Pota, "Best practices for portfolio optimization by quantum computing, experimented on real quantum devices," *Sci. Rep.*, vol. 13, 2023, Art. no. 19434, doi: 10.1038/s41598-023-45392-w.

[10] P. K. Barkoutsos, G. Nannicini, A. Robert, I. Tavernelli, and S. Woerner, "Improving variational quantum optimization using CVaR," *Quantum*, vol. 4, p. 256, Apr. 2020, doi: 10.22331/q-2020-04-20-256.

[11] M. J. Best, *Portfolio Optimization*. Boca Raton, FL, USA: CRC Press, 2010. [Online]. Available: https://www.routledge.com/Portfolio-Optimization/Best/p/book/9781420085846

[12] M. E. Stucke, "Behavioral economists at the gate: Antitrust in the twenty-first century," *Loy. U. Chi. LJ*, vol. 38, no. 3, pp. 513–592, 2007. [Online]. Available: https://loyola-chicago-law-journal.scholasticahq.com/article/76753-behavioral-economists-at-the-gate-antitrust-in-the-twenty-first-century

[13] S. K. Mehra, "Competition law for a post-scarcity world," *Texas A&M Law Rev.*, vol. 4, no. 1, pp. 1–39, 2016, doi: 10.37419/LR.V4.I1.1.

[14] A. Naik, E. Yeniaras, G. Hellstern, G. Prasad, and S. K. L. P. Vishwakarma, "From portfolio optimization to quantum blockchain and security: A systematic review of quantum computing in finance," Jun. 2023, *arXiv:2307.01155*, doi: 10.48550/arXiv.2307.01155.

[15] D. J. Egger et al., "Quantum computing for finance: State-of-the-art and future prospects," *IEEE Trans. Quantum Eng.*, vol. 1, 2020, Art. no. 3101724, doi: 10.1109/TQE.2020.3030314.

[16] G. Cornuejols, M. L. Fisher, and G. L. Nemhauser, "Exceptional paper–location of bank accounts to optimize float: An analytic study of exact and approximate algorithms," *Manage. Sci.*, vol. 23, pp. 789–810, Apr. 1977, doi: 10.1287/mnsc.23.8.789.

[17] M. Tisi, A. Kalra, and F. Qureshi, "Portfolio asset identification using graph algorithms on a quantum annealer," *IUP J. Comput. Sci.*, vol. XIII, no. 4, Oct. 2019, pp. 52–71. [Online]. Available: https://ssrn.com/abstract=3794998

[18] G. Rosenberg and M. Rounds, "Long-short minimum risk parity optimization using a quantum or digital annealer," White Paper, 2018. [Online]. Available: https://1qbit.com/whitepaper/long-short-minimum-risk-parity-optimization-quantum-or-digital-annealer/

[19] H. Markowitz, "Portfolio selection*," *J. Finance*, vol. 7, no. 1, pp. 77–91, 1952, doi: 10.1111/j.1540-6261.1952.tb01525.x.

[20] M. Hodson, B. Ruck, H. Ong, D. Garvin, and S. Dulman, "Portfolio rebalancing experiments using the quantum alternating operator ansatz," Nov. 2019. *arXiv:1911.05296*, doi: 10.48550/arXiv.1911.05296.

[21] A. Gilliam, S. Woerner, and C. Gonciulea, "Grover adaptive search for constrained polynomial binary optimization," *Quantum*, vol. 5, Apr. 2021, Art. no. 428, doi: 10.22331/q-2021-04-08-428.

[22] G. Rosenberg, P. Haghnegahdar, P. Goddard, P. Carr, K. Wu, and M. L. de Prado, "Solving the optimal trading trajectory problem using a quantum annealer," *IEEE J. Sel. Topics Signal Process.*, vol. 10, pp. 1053–1060, 2016, doi: 10.1145/2830556.2830563.

[23] S. Mugel et al., "Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks," *Phys. Rev. Res.*, vol. 4, Jan. 2022, Art. no. 013006, doi: 10.1103/PhysRevResearch.4.013006.

[24] E. Grant, T. S. Humble, and B. Stump, "Benchmarking quantum annealing controls with portfolio optimization," *Phys. Rev. Appl.*, vol. 15, Jan. 2021, Art. no. 014012, doi: 10.1103/PhysRevApplied.15.014012.

[25] D. Venturelli and A. Kondratyev, "Reverse quantum annealing approach to portfolio optimization problems," *Quantum Mach. Intell.*, vol. 1, pp. 17–30, May 2019, doi: 10.1007/s42484-019-00001-w.

[26] H. Markowitz, *Portfolio Selection: Efficient Diversification of Investments*. New Haven, CT, USA: Yale Univ. Press, 1959. [Online]. Available: https://www.jstor.org/stable/j.ctt1bh4c8h

[27] I. Kerenidis, A. Prakash, and D. Szilágyi, "Quantum algorithms for portfolio optimization," in *Proc. 1st ACM Conf. Adv. Financial Technol.*, New York, NY, USA, 2019, pp. 147–155, doi: 10.1145/3318041.3355465.

[28] I. Kerenidis and A. Prakash, "A quantum interior point method for LPs and SDPs," *ACM Trans. Quantum Comput.*, vol. 1, pp. 1–32, Oct. 2020, doi: 10.1145/3406306.

[29] P. Rebentrost and S. Lloyd, "Quantum computational finance: Quantum algorithm for portfolio optimization," Nov. 2018, *arXiv:1811.03975*, doi: 10.48550/arXiv.1811.03975.

[30] N. Slate, E. Matwiejew, S. Marsh, and J. B. Wang, "Quantum walk-based portfolio optimisation," *Quantum*, vol. 5, Jul. 2021, Art. no. 513, doi: 10.22331/q-2021-07-28-513.

[31] S. Woerner and D. J. Egger, "Quantum risk analysis," *npj Quantum Inf.*, vol. 5, pp. 1–8, Feb. 2019, doi: 10.1038/s41534-019-0130-6.

[32] D. J. Egger, R. García Gutiérrez, J. C. Mestre, and S. Woerner, "Credit risk analysis using quantum computers," *IEEE Trans. Comput.*, vol. 70, no. 12, pp. 2136–2145, Dec. 2021, doi: 10.1109/TC.2020.3038063.

[33] N. S, G. B, N. S, and A. B. H, "The potential of quantum techniques for stock price prediction," Aug. 2023. *arXiv:2308.13642*, doi: 10.48550/arXiv.2308.13642.

[34] P. Rebentrost, M. Mohseni, and S. Lloyd, "Quantum support vector machine for Big Data classification," *Phys. Rev. Lett.*, vol. 113, Sep. 2014, Art. no. 130503, doi: 10.1103/PhysRevLett.113.130503.

[35] S. A. Vavasis, "Quadratic programming is in NP," *Inf. Process. Lett.*, vol. 36, no. 2, pp. 73–77, 1990, doi: 10.1016/0020-0190(90)90100-C.

[36] A. J. King and D. L. Jensen, "Linear-quadratic efficient frontiers for portfolio optimization," *Appl. Stochastic Models Data Anal.*, vol. 8, no. 3, pp. 195–207, 1992, doi: 10.1002/asm.3150080309.

[37] M. J. Best and J. K. Kale, "Quadratic programming for large-scale portfolio optimization," in *Financial Services Information Systems*, Auerbach Publications, 2000, pp. 531–548. [Online]. Available: https://www.taylorfrancis.com/chapters/edit/10.1201/9780203997611-33/quadratic-programming-large-scale-portfolio-optimization-michael-best-jivendra-kale

[38] J. Sen and A. Dutta, "A comparative study of hierarchical risk parity portfolio and eigen portfolio on the NIFTY 50 stocks," in *Computational Intelligence and Data Analytics*. Berlin, Germany: Springer, 2023, pp. 443–460, doi: 10.1007/978-981-19-3391-2_34.

[39] A. Dezhkam and M. T. Manzuri, "Forecasting stock market for an efficient portfolio by combining XGBoost and Hilbert-Huang," *Eng. Appl. Artif. Intell.*, vol. 118, 2023, Art. no. 105626, doi: 10.1016/j.engappai.2022.105626.

[40] C. Garcia and F. Gould, "Survivorship bias," *J. Portfolio Manage.*, vol. 19, no. 3, pp. 52–56, Spring, 1993, doi: 10.3905/jpm.1993.409445.

[41] G. Daniel, D. Sornette, and P. Woehrmann, "Look-ahead benchmark bias in portfolio performance evaluation," *J. Portfolio Manage.*, vol. 36, no. 1, pp. 121–130, 2009, doi: 10.3905/JPM.2009.36.1.121.

[42] M. Owhadi-Kareshk and P. Boulanger, "Portfolio optimization on classical and quantum computers using PortFawn," 2021, *arXiv:2112.08998*, doi: 10.48550/arXiv.2112.08998.

[43] "BME bolsas y mercados españoles," [Online]. Available: https://www.bolsasymercados.es/esp/Home

[44] "Qiskit: An open-source framework for quantum computing," 2017. [Online]. Available: https://qiskit.org/

[45] Yahoo Finanzas, "Componentes de IBEX 35 ... (^IBEX)," [Online]. Available: https://es.finance.yahoo.com/quote//%5EIBEX/components?p=%5EIBEX

[46] P. Morissette, "bt - flexible backtesting for Python," 2015. [Online]. Available: https://pmorissette.github.io/bt/

[47] J. Pérez O et al., "Comparison and selection of exact and heuristic algorithms," in *Computational Science and Its Applications–ICCSA* (Lecture Notes in Computer Science Series), A. Laganá, M. L. Gavrilova, V. Kumar, Y. Mun, C. J. K. Tan, and O. Gervasi, Eds., Berlin, Heidelberg: Springer, 2004, pp. 415–424. doi: 10.1007/978-3-540-24767-8_43

[48] J. Dahl, L. Vandenberghe, and M. Andersen, "CVXOPT - Python software for convex optimization," 2004. [Online]. Available: https://cvxopt.org/

[49] Qiskit, "Simulating molecules using VQE," Accessed: Dec. 29, 2022. [Online]. Available: https://github.com/Qiskit/textbook/blob/main/notebooks/ch-applications/vqe-molecules.ipynb

[50] N. Sá, I. S. Oliveira, and I. Roditi, "Towards solving the BCS Hamiltonian gap in near-term quantum computers," *Results Phys.*, vol. 44, Jan. 2023, Art. no. 106131, doi: 10.1016/j.rinp.2022.106131.

[51] J. Gambetta and S. Sheldon, "Cramming more power into a quantum device IBM research blog," 2019. Accessed: Dec. 4, 2023. https://www.ibm.com/blogs/research/2019/03/power-quantum-device/

[52] R. Mansini and M. G. Speranza, "An exact approach for portfolio selection with transaction costs and rounds," *IIE Trans.*, vol. 37, no. 10, pp. 919–929, 2005, doi: 10.1080/07408170591007821.

[53] M. T. Faber, "A quantitative approach to tactical asset allocation," *J. Wealth Manage.*, vol. 9, pp. 69–79, Jan. 2007, doi: 110.3905/jwm.2007.674809.

[54] M. I. Ahmed, W. Ghohui, M. Hasan, A. Sattar, M. Ahmed, and R. U. Rehman, "Performance of moving average investment timing strategy in U.K. stock market: Individual stocks versus portfolios," *J. Econ. Social Stud.*, vol. 7, no. 2, 2018, doi: 10.14706/JECOSS17722.

[55] W. F. Sharpe, "Capital asset prices: A theory of market equilibrium under conditions of risk*," *J. Finance*, vol. 19, no. 3, pp. 425–442, 1964, doi: 10.1111/j.1540-6261.1964.tb02865.x.

[56] D. H. Bailey and M. Lopez de Prado, "The sharpe ratio efficient frontier," *J. Risk*, vol. 15, no. 2, pp. 3–44, 2012, doi: 10.2139/ssrn.1821643.

[57] S. Farinelli, M. Ferreira, D. Rossello, M. Thoeny, and L. Tibiletti, "Beyond Sharpe ratio: Optimal asset allocation using different performance ratios," *J. Bank. Finance*, vol. 32, no. 10, pp. 2057–2063, 2008, doi: 10.1016/j.jbankfin.2007.12.026.

[58] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," Nov. 2014. *arXiv:1411.4028*, doi: 10.48550/arXiv.1411.4028.

Ginés Carrascal was born in Salamanca, Spain, in 1975. He received the M.Sc. degree in physics from the University of Salamanca, Salamanca, Spain, in 1999.

Since 2000, he has been an Architect with IBM Consulting Spain, getting involved with quantum computing in 2017, and currently acting as IBM Quantum Computational Scientist, Quantum Technical Ambassador, and Qiskit Advocate. He has become IBM Certified Associate Developer—Quantum Computation using Qiskit v0.2X in 2021. Since 2014, he has been an Adjunct Professor of computer science with Universidad Carlos III de Madrid, Getafe, Spain, and from 2018, an Adjunct Professor with Universidad Complutense de Madrid, Madrid, Spain, at the computer science departmental section of the Mathematics Faculty. His research interests include artificial intelligence and the application of quantum computing to optimization problems, especially but not only in the field of banking and financial services.

Dr. Carrascal was the recipient of two Outstanding Technical Achievement Awards (OTAA) and received the highest IBM technical award, in 2012 and 2016.

Paula Hernamperez was born in Valladolid, Spain in 1996. She received the M.Sc degree in mathematical engineering from the Complutense University of Madrid, Madrid, Spain, in 2021.

Since 2019 she has been a Researcher with the CARTIF Technology Centre's Energy Division, and from 2022, Team Leader participating in national and international projects related to climatic change prevention, new battery technologies development, energy efficiency, optimal control of smart grids, and renewable energy sources penetration. Her current research interests include machine learning, optimization problems resolution new methodologies and data sciences, and the application of new technologies to the field of energy and energy markets.

Guillermo Botella (Senior Member, IEEE) received the M.Sc. degree in physics (fundamental), the M.Sc. degree in electronic engineering, and the Ph.D. degree in computer engineering in 1998, 2001, and 2007, respectively, all from the University of Granada, Granada, Spain.

He was a Research Fellow funded by the EU with the University of Granada, and the Vision Research Laboratory with University College London, London, U.K. After that, he joined as an Assistant Professor with the Department of Computer Architecture and Automation, Complutense University of Madrid, Madrid, Spain, where he is currently an Associate Professor (Full Professor habil.). From 2008 to 2012, he has performed research stays and he was Visiting Professor with the Department of Electrical and Computer Engineering, Florida State University, Tallahassee, FL, USA. His current research interests include image and video processing for VLSI, FPGAs, GPGPUs, embedded systems, and novel computing paradigms, such as analog and quantum computing.

Alberto del Barrio (Senior Member, IEEE) received the Ph.D. degree in computer science from the Complutense University of Madrid (UCM), Madrid, Spain, in 2011.

He has performed stays at Northwestern University, the University of California at Irvine, and the University of California at Los Angeles. Since 2021, he is an Associate Professor (tenure-track, civil-servant) of computer science with the Department of Computer Architecture and System Engineering, UCM. He has been the Principal Investigator (PI) of the PARNASO project, funded by the Leonardo Grants program by Fundación BBVA, and currently, he is the PI of the ASIMOV project, funded by the Spanish MICINN, which includes a work package to research on the applicability of quantum computing. His main research interests include design automation, arithmetic, analog/quantum computing, and their application to the field of artificial intelligence.

Dr. Barrio has been an ACM Senior Member since December 2020.